

Case study: tic-bot-toe w 6 dni

Jak z 30-minutowej rozmowy w samochodzie powstała działająca, publiczna aplikacja webowa — i czego ta historia uczy o pracy z agentami.

Zeszyt 2 z 4 · Pakiet „Jak powstała aplikacja tic-bot-toe”

Wszystkie liczby i cytaty pochodzą z historii projektu (git, dokumenty repozytorium, zapis rozmowy koncepcyjnej).

Stan na 18.07.2026

Czego się tu nauczysz

To nie jest opowieść „patrzcie, jakie AI mądre”. To rozbiór **metody**, którą możesz powtórzyć. Zobaczysz:

- jak wyglądała droga od pomysłu do produkcji, dzień po dniu,
- kto (który człowiek i który model) za co odpowiadał,
- pięć zasad, dzięki którym całość się nie zawaliła,
- co poszło nie tak — i jak metoda to wyłapała.

1. Punkt zero: 30 minut w samochodzie

Cała koncepcja aplikacji powstała w **jednej rozmowie** z Claude (w trybie Cowork), prowadzonej **z telefonu, jako pasażer w samochodzie** — w około **30 minut**. Nie było dokumentu na start, prezentacji ani tygodni analiz. Był pomysł i rozmowa, która ten pomysł wykuła w konkretną specyfikację.

Impuls był prosty:

„Zaplanuj i zaprojektuj aplikację webową [...] która będzie umożliwiała wybór pomiędzy różnymi modelami, które będą grały przeciw sobie w kółko i krzyżyk. [...] Aplikacja powinna być odporna na boty [...]. Ponadto ja nie chcę płacić za użycie z własnych pieniędzy.”

Z tego jednego akapitu, w dialogu, wyrosła kompletna specyfikacja. To pierwszy morał: **dobra koncepcja nie wymaga wielu godzin — wymaga dobrych pytań i szczerych odpowiedzi**.

2. Jak rosła specyfikacja (faza koncepcji)

Rozmowa nie wypluła gotowca. Specyfikacja **rosła warstwami** — każde pytanie autora dokładało kolejną. To wzorzec „researchu przez dialog”.

Wersja	Co dorzucił autor	Co przybyło
v1	pomysł wyjściowy	kółko i krzyżyk, klucz użytkownika (nie płacę za graczy), odporność na boty, parser ruchów z zabezpieczeniami
—	„są apki, które to robią?”	analiza konkurencji (przeszukanie sieci) → wniosek: ta nisza jest wolna

Wersja	Co dorzucił autor	Co przybyło
v2	„rankingi + statki z wyborem planszy”	Elo, walidacja partii, statki 6×6/8×8/10×10, architektura wielogrowa
v3	„hostuję na własnym VPS z Postgresem”	zejście z chmury na Node + Hono + PostgreSQL + Docker
v4	„dodaj telemetry, wykresy, wartość edukacyjną”	pomiary (czas, tokeny, koszt), wykresy, moduł edukacyjny; podział na rdzeń i moduły

Na koniec autor poprosił o **prompt startowy** dla Claude Code — jedno polecenie, które uruchomiło budowę. Cała ta rozmowa to około **11 wymian**, z czego ~6 budowało specyfikację.

WSKAZÓWKA

Ważny fakt: dostarczona specyfikacja v4 jest **identyczna** z tą, która została w repozytorium po zakończeniu projektu. Spec napisano **raz, z góry** — i zbudowano według niej bez przepisywania wymagań w trakcie.

3. Podział ról: kto co robił

Kod powstał w narzędziu **Claude Code**, w którym pracowały modele AI. Człowiek prowadził. Oto twardy podział (liczby z historii git):

Rola	Wykonawca	Ślad
Koncepcja, research, specyfikacja	Człowiek + Claude (Cowork)	zapis rozmowy koncepcyjnej
Rdzeń aplikacji + większość funkcji	Claude Code + Opus 4.8 (1M)	49 commitów
Poprawki, code-review	Claude Code + Opus 4.8	6 commitów
Pakiet funkcji „WOW”	Claude Code + Claude Fable 5	16 commitów
Decyzje, briefy, recenzja, weryfikacja	Człowiek	autor wszystkich commitów

Razem: **71 commitów w 6 dni**. Kod pisany ręcznie przez człowieka: **0 linii**. To nie znaczy, że człowiek nic nie robił — robił rzecz najważniejszą: **decydował i sprawdzał**.

4. Oś czasu: 6 dni od pomysłu do produkcji

WSKAZOWKA

Dzień 1 (12.07) — rdzeń w jeden dzień. Od pustego repozytorium do grywalnej areny: silniki gier, obsługa modeli, backend, rankingi Elo, wersja do wdrożenia. 14 commitów.

Dzień 2 (13.07) — hardening i wyjście na świat: poprawki bezpieczeństwa, dostępność, testy w prawdziwej przeglądarce, wersja angielska, a na koniec **produkcja na żywo pod adresem ticbottoe.lol**.

Dzień 3 (14.07) — drobna poprawka.

Dni 4–5 (16–17.07) — dwie nowe gry: Sudoku Duel i słowna bitwa (odpowiednik Scrabble), z własnymi silnikami i słownikami.

Dni 5–6 (17–18.07) — sześć modułów „WOW”, które zamieniają benchmark w spektakl: podgląd toku myślenia modeli, ranking i muzeum ich wpadek, mapy zachowań, tryb „kto jest botem?”, demo grające w przeglądarce i pojedynki promptów.

Rdzeń grywalny **pierwszego dnia**, publiczna produkcja **drugiego**. Reszta to rozbudowa na już działającym fundamencie.

5. Dlaczego to zadziałało – pięć zasad metody

To jest sedno zeszytu. Te zasady są przenośne — użyjesz ich w Zeszycie 4.

1. Specyfikacja jako źródło prawdy

Jeden dokument (`SPEC.md`) opisywał, co ma powstać — i był w repozytorium, więc agent mógł do niego **wracać** w trakcie pracy, zamiast polegać na urywanej pamięci. Prompt startowy mówił wprost:

*„Przeczytaj CAŁĄ specyfikację przed napisaniem pierwszej linii kodu i trzymaj się jej [...].
Zaczynij od etapu 1.”*

2. Praca etapami z „definicją ukończenia”

Duże zadanie pocięto na etapy. Po **każdym** etapie musiało być zielono, zanim ruszył kolejny:

„Po każdym etapie: `pnpm test` musi być zielony [...]. Nie przechodź do kolejnego etapu z czerwonymi testami.”

3. Dziennik decyzji zamiast przerywania pytaniami

Gdy specyfikacja czegoś nie rozstrzygała, agent miał **sam** podjąć rozsądną decyzję, zapisać ją jednym zdaniem w `DECISIONS.md` i jechać dalej — a zatrzymać się tylko, gdyby coś łamało wymagania nadrzędne. Dzięki temu praca miała ciągłość i była audytowalna.

4. Testy pisane razem z kodem

Nie „testy na końcu”. Każdy etap dowoził kod i testy, ze szczególnym naciskiem na logikę gier. To one wyłapywały błędy, zanim urosły.

5. Code-review jako osobny krok

Po większych etapach przychodził osobny przegląd kodu i osobne commity „poprawki z code-review”. Pisanie i krytykowanie to dwie różne czynności — rozdzielenie ich poprawia jakość.

6. Co poszło nie tak (i dlaczego to dobra wiadomość)

Metoda nie polega na tym, że nic się nie psuje. Polega na tym, że **wpadki wychodzą szybko**. Trzy przykłady:

Reguła, która po cichu psuła produkt. Założono, że „ruch modelu szybszy niż 3 sekundy = podejrzany”. W praktyce uczciwe, szybkie modele były przez to odrzucane, a ranking po cichu przyjmował tylko wolne. Wyłapano to dopiero **pomiarem na żywo**. Wniosek z dziennika decyzji jest bezcenny:

„Testy tego nie łapały, bo wpisywały `LatencyMs: 4000` — były pisane pod regułę, nie pod rzeczywistość.”

To lekcja o granicach testów: test potwierdza założenie, ale nie sprawdza, czy założenie jest prawdziwe. Dlatego **weryfikacja na żywo** jest osobnym krokiem.

Świadome odkładanie funkcji. Część efektów (np. animacja „pisania na żywo”) świadomie odłożono, zamiast dłużej je połowicznie — i dokończono dopiero, gdy powstał pod nie właściwy fundament. Dojrzały projekt to też umiejętność powiedzenia „jeszcze nie teraz”.

Poprawki bezpieczeństwa łatanie wcześniej. Zamiast czekać do końca, luki (np. sposób zapisywania wyników do rankingu) domykaną na bieżąco, osobnymi commitami.

7. Zaskakujący morał: model odradzał, człowiek zdecydował

W rozmowie koncepcyjnej padło pytanie „czy to ma sens?”. Odpowiedź modelu była szczerą i... **odradzającą** pełny zakres:

„Nie buduj v4. Zbuduj rdzeń w wersji minimalnej [...], wypuść [...] i zobacz, czy ktokolwiek poza Tobą rozegra 20 partii.”

Autor **świadomie** poszedł szerzej — powstały 4 gry zamiast 2 i sześć dodatkowych modułów. I to jest właściwa nauka o rolach: **agent doradza, człowiek decyduje**. Model dał rozsądną, ostrożną rekomendację; człowiek miał inny cel (portfolio, materiał edukacyjny, demonstracja metody) i miał prawo ją napisać. Co ciekawe, ta sama rozmowa przewidziała ten zeszyt:

„Sam proces budowy specyfikacją przez Claude Code to case study wart osobnego posta.”

8. Co konkretnie powstało

tic-bot-toe (na żywo: ticbottoe.lol) — arena, w której modele językowe grają w gry logiczne przeciw sobie i ludziom, ucząc przez grę:

- **4 gry**: kółko i krzyżyk, statki, Sudoku Duel, słowna bitwa,
- **2 tryby**: model kontra model (ogładasz) i człowiek kontra model (grasz),
- **rankingi Elo** liczone i sprawdzane po stronie serwera (nie na słowo klienta),
- **moduły edukacyjne**: podgląd „toku myślenia”, muzeum wpadek modeli, mapy ich zachowań, tryb „kto jest botem?”,
- **model kosztowy**, w którym właściciel nie płaci za rozgrywki graczy.

9. Pięć rzeczy do zapamiętania

1. Dobra **koncepcja** bywa kwestią 30 minut dobrej rozmowy — nie tygodni.
2. **Specyfikacja** napisana raz, z góry, jest kotwicą całego projektu.
3. Praca **etapami** z zielonymi testami po każdym kroku chroni przed chaosem.
4. **Weryfikacja na żywo** wyłapuje to, czego testy z definicji nie złapią.
5. **Agent doradza, człowiek decyduje** — autorstwo to kierunek, nie klawiatura.

OK - PUNKT KONTROLNY

Umiesz wskazać, **który** z pięciu elementów metody (sekcja 5) najbardziej zapobiega „budowaniu wszystkiego naraz”? Jeśli tak — rozumiesz, dlaczego ten projekt się udał, i możesz przejść do Zeszytu 3, gdzie ustawimy Ci środowisko do powtórzenia tego samemu.